How to identify functional requirements

Tutorial

What are functional requirements?

- Defined as the specific behaviours, features, and functions that a system must perform to meet the user needs
- Define what the application must be able to do/achieve

Steps

- Step 1: Define the goal of the system
- Step 2: Identify the core features
- Step 3: Identify constraints
- Step 4: Identify Edge Cases

Functional Requirements for a Calculator application

Step 1: Define the goal of the system

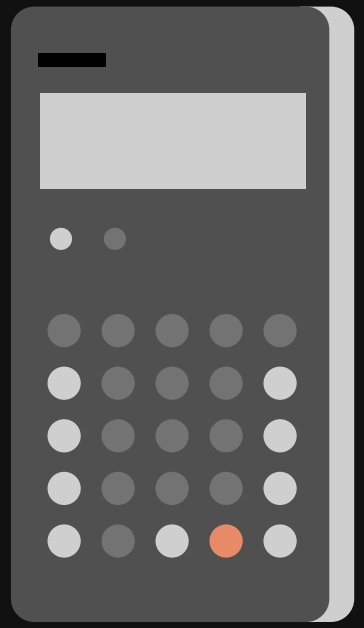
Calculator application requirements

The calculator application should be able to perform basic arithmetic functions (addition, subtraction, multiplication and division) on a set of hexadecimal numbers.

The calculator's only restriction is that it will accept inputs up to 2 digits, return answers up to 4 digits, and not return any negative answers or answers with decimal places. (Note that you can do all these on the client side, but for the sake of this assignment, make sure the code under the hood supports this as well.)

Step 2: Identify the core features

- Accept input
- Perform basic arithmetic functions:
  - Perform addition
  - Perform subtraction
  - Perform multiplication
  - Perform division
- Return and display output



Step 3: Identify constraints

- Input must be 1 or 2 digits
- Input must contain valid hexadecimal numbers
- Input must include a valid operator (+, -, /, *)
- Output must be less than 5 digits
- Output must not be negative
- Output must not contain decimal places

Step 4: Identify Edge Cases

- Invalid hexadecimal input must return an error
- Missing or invalid operator must return an error
- Input exceeding 2 digits must return an error
- Division by zero must return an error
- Output that is greater than 4 digits will return an error
- Output that is negative will return an error
- Output with decimal places will return an error

Functional Requirements

- Accept input
- Perform basic arithmetic functions:
  - Perform addition
  - Perform subtraction
  - Perform multiplication
  - Perform division
- Return and display output
- Input must be 1 or 2 digits
- Input must contain valid hexadecimal numbers
- Input must include a valid operator (+, -, /, *)
- Output must be less than 5 digits (<=4)
- Output must not be negative
- Output must not contain decimal places
- Invalid hexadecimal input must return an error
- Missing or invalid operator must return an error
- Input exceeding 2 digits must return an error
- Division by zero must return an error
- Output that is greater than 4 digits will return an error
- Output that is negative will return an error
- Output with decimal places will return an error

Next Step

Categorise the functional requirements